গভীর ডেক · D1

ডেটাবেস ডিজাইন

৩১ টেবিল · ৩০১ কলাম · ৭৪ সম্পর্ক · ২৩ মাইগ্রেশন-এ না, ১১টিতে — স্কুলের কাগজ কীভাবে টেবিল হলো

মানচিত্র

এই ৩০ স্লাইডে কী আছে

চারটি ব্লক — প্রতিটি ব্লকের শেষে একটি সংখ্যা প্রমাণ হিসেবে

১. আকার ও নাম

স্লাইড ৩–৫ · ১১ অ্যাপ · ৩১ মডেল · ৩০১ কলাম · নামকরণের ৪ নিয়ম

প্রমাণ: ইন্ট্রোস্পেকশন আউটপুট

২. সীমানা ও সম্পর্ক

স্লাইড ৬–১১ · workspace দেয়াল · ইউজার মডেল · academ কশিরা

প্রমাণ: ২৩ টেবিলে workspace কলাম

৩. নিয়ম ডেটাবেজে

স্লাইড ১২–১৯ · ১৩ unique_together · ৮ শর্তসাপেক্ষ unique · ২ check · ১৩ ইনডেক্স

প্রমাণ: কনস্ট্রেইন্টের নাম

৪. আসল ব্যবহার

স্লাইড ২০–৩০ · হাজিরা · নম্বর · টাকা · সময় · কোয়েরি · মাইগ্রেশন · ব্যাকআপ · ফাঁক

প্রমাণ: ১০৩ টেস্ট · ৫৭ select_related

শেষ তিন স্লাইড ইচ্ছাকৃতভাবে **অস্বস্তিকর**: যা মাপা হয়নি, যা EXPLAIN করা হয়নি, যা এখনো টেবিলে নেই।

শিক্ষকের নোট: এই স্লাইডটি ৩০ সেকেন্ডেই ছাড়ুন — উদ্দেশ্য কেবল শ্রোতাকে জানানো যে কোথায় কী পাবেন। শিক্ষার্থীদের বলুন কোন ব্লকটি তাদের সবচেয়ে দরকার, তার উপর জোর দেবেন।

উৎস

৩১ টেবিল কোথা থেকে এল

টেবিল বানানো হয় না — বিদ্যালয়ের কাগজ থেকেই টেবিল বেরোয়

বিদ্যালয়ে যা আছে	যে প্রশ্নটা জিজ্ঞেস করলাম	যে টেবিল হলো	কেন এটাই সঠিক
ভর্তি খাতা	"এই ছেলেটি কে, আলাদা করে চেনার উপায় কী?"	students.StudentProfile	নাম বদলাতে পারে, `admission_no` নয়
শ্রেণি খাতা	"ছাত্রটি কোন শ্রেণিতে, কোন সেশনে?"	students.Enrollment	শ্রেণি বদল ইতিহাস, মুছে ফেলা নয়
হাজিরা রেজিস্টার	"একই দিনে একই ক্লাসে দুবার লেখা যাবে?"	attendance.AttendanceRecord	প্রতিটি ঘর = এক সারি, unique দিয়ে আটকানো
নম্বরের তালিকা	"অনুপস্থিত আর শূন্য কি এক?"	assessments.Mark	তাই `is_absent` আলাদা কলাম
সিলেবাস ও ডায়েরি	"অধ্যায় → টপিক, কে শেষ করল?"	lessons.Chapter · Topic	দুই ধাপের গাছ, এক টেবিলে গুঁজে দিইনি
হিসাবের খাতা	"কোন প্ল্যানে কত টাকা, কত দিন?"	billing.Plan · Subscription	প্ল্যান বদলালেও পুরনো ইনভয়েস অটুট

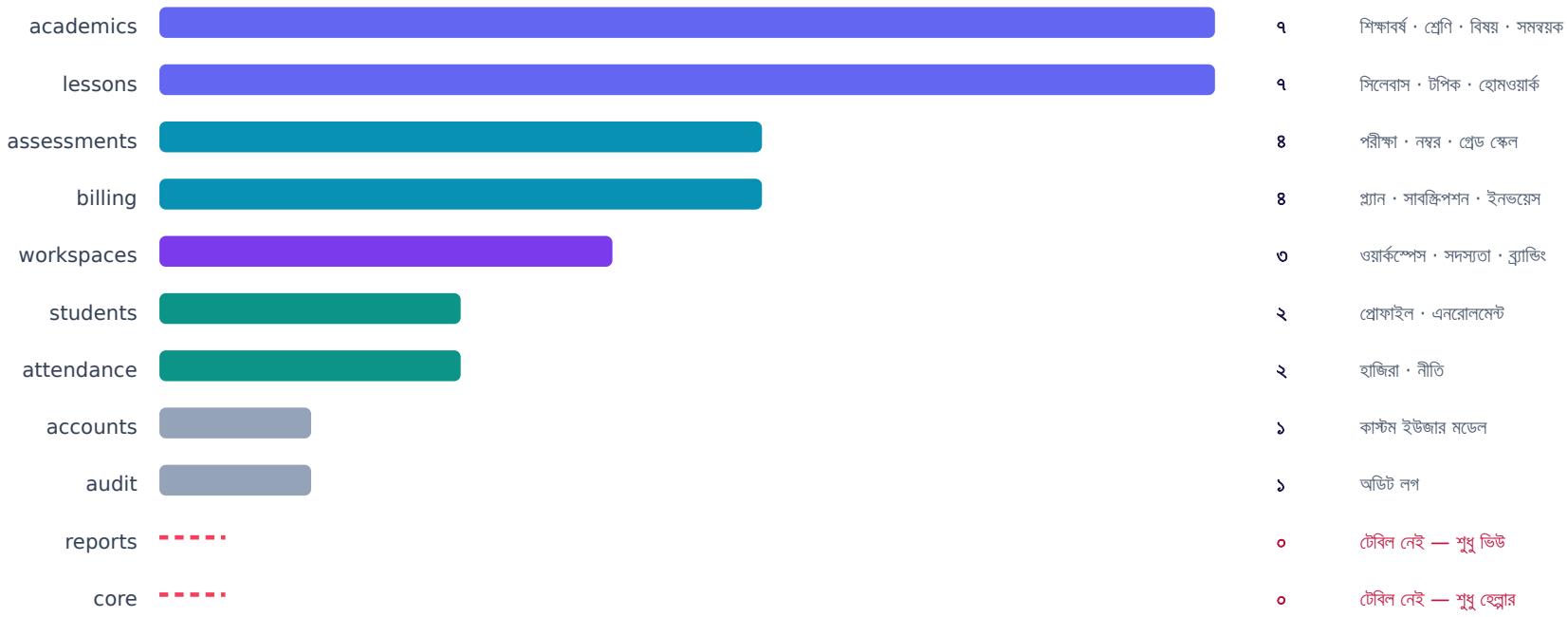
নিয়ম: **কাগজে যে ঘরটি আলাদা করে লেখা হয়, ডেটাবেজে সেটি আলাদা কলাম।** যে ঘর আলাদা লেখা হয় না, সেটি নতুন টেবিল।

শিক্ষকের নোট: বোর্ডে একটি হাজিরা রেজিস্টারের পাতা এঁকে দেখান — তারিখ, শ্রেণি, ৪০টি নাম। তারপর জিজ্ঞেস করুন: "এই পাতায় কতটি আলাদা তথ্য?" উত্তর ৪০ নয়, চারটি: তারিখ, শ্রেণি, বিষয় আর প্রতিটি ছাত্রের একটি অবস্থা।

আকার

১১ অ্যাপে ৩১ মডেল — কে কতটা ভারী

নামের পাশে সংখ্যা = মডেল সংখ্যা; মোট ৩০১ কলাম, ৭৪ সম্পর্ক



reports ও **core**-এ শূন্য মডেল — ইচ্ছাকৃত। রিপোর্ট অন্য অ্যাপের টেবিল পড়ে, নিজের টেবিল বানালে ডেটা দুই জায়গায় থাকত।

শিক্ষকের নোট: যে দুটি অ্যাপে মডেল নেই, সেখানেই থামুন। প্রশ্ন করুন: "রিপোর্টের নিজের টেবিল থাকলে কী সমস্যা হতো?" উত্তর — একই হাজিরা দুই জায়গায়, একটি বদলালে অন্যটি পুরনো।

নামকরণ

চারটি নামকরণ-নিয়ম, ৩১ টেবিলে অক্ষরে অক্ষরে

নাম ভুল হলে পরের ডেভেলপার প্রতিটি কোয়েরিতে ভুল করবে

নিয়ম	উদাহরণ	কেন এভাবে
মডেল একবচন, টেবিল বহুবচন	`ClassGroup` → `academics_classgroup`	কোডে একটি সারি বোঝায়, ডেটাবেজে অনেক সারি — দুই জায়গায় দুই বচন স্বাভাবিক
কলামে `_id` থাকে, মডেলে থাকে না	`class_group` লিখি, DB-তে `class_group_id`	`obj.class_group` = object, `obj.class_group_id` = সংখ্যা — এক নামে দুই জিনিস নয়
`related_name` সবসময় দেওয়া	`related_name="enrollments"`	উল্টো দিকের নাম না দিলে Django `enrollment_set` বানায় — পড়তে খারাপ, দুই অ্যাপে সংঘর্ষও হয়
`related_name="+"` = উল্টো দিক দরকার নেই	`recorded_by`, `entered_by`, `published_by`	ইউজার থেকে "সে কী কী হাজিরা লিখেছে" তালিকা কেউ চায় না; অকারণ রিভার্স-কোয়েরি বন্ধ
শর্ত বোঝায় এমন নাম	`is_archived` · `is_published` · `is_absent`	`status` নামের বুলিয়ানে অর্থ হারায়; `is_` উপসর্গে সত্য/মিথ্যা স্পষ্ট

বাস্তব ফল: `related_name` থাকার কারণে এক কোয়েরিতেই `class_group.enrollments.filter(is_active=True)` লেখা যায় — তিনটি ভিন্ন ভিউতে একই লাইন, তিনবার ভিন্ন কোয়েরি নয়।

শিক্ষকের নোট: নামকরণের নিয়ম মুখস্থ করাবেন না — নিজের প্রজেক্টের একটি মডেল দেখিয়ে প্রতিটি নিয়ম কোথায় লঙ্ঘিত হয়েছে খুঁজতে বলুন। এরপর শিক্ষার্থীদের বলুন নিজের প্রজেক্টে একটি মডেলের সব `related_name` লিখে ফেলতে।

সীমানা

`workspace` কলাম — ২৩ টেবিলে আছে, ৮-এ নেই, কারণ লেখা আছে

একটি ডেটাবেজে বহু বিদ্যালয়; দেয়ালটা কোথায়

শ্রেণি	সংখ্যা	কোনগুলো	কেন
নিজের `workspace` কলাম আছে	২৩	ClassGroup · Exam · AttendanceRecord · Homework · Invoice …	প্রতিটি লিস্ট-কোয়েরি এক লাইনেই সীমা টানে
মূল/গ্লোবাল টেবিল	৩	`Workspace` · `User` · `Plan`	এরা নিজেরাই সীমানার বাইরে — এক ইউজার বহু বিদ্যালয়ে
প্যারেন্ট দিয়ে সীমানা	৫	`Chapter` → Syllabus · `Topic` → Chapter · `Submission` → Homework · `Mark` → Exam · `GradeBand` → GradeScale	কলাম যোগ করলে দুই জায়গায় সত্য থাকত, একদিন মিলত না

দাম: এই পাঁচটি টেবিলে `filter(workspace=…)` লেখা যায় না — `filter(exam__workspace=…)` লিখতে হয়। ভুল করলে কোয়েরি চলে, কিন্তু অন্য বিদ্যালয়ের ডেটাও আসতে পারে।

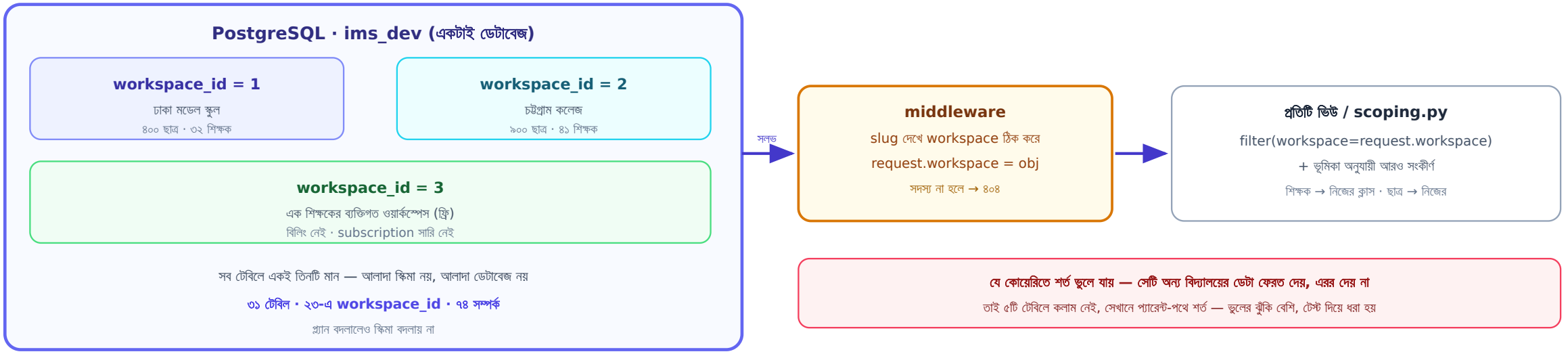
প্রতিরোধ: ভিউ কখনো সরাসরি ফিল্টার লেখে না — `apps/core/scoping.py`-র হেল্পার ডাকে, যেখানে সীমাটা একবার লেখা।

শিক্ষকের নোট: এখানেই বলুন — "মাল্টি-টেন্যান্ট" শব্দটা জাদু নয়। এটি কেবল একটি কলাম আর একটি অভ্যাস: প্রতিটি কোয়েরিতে ওই কলামের শর্ত। অভ্যাস ভাঙলে ডেটা মিশে যায়, আর মিশে গেলে ফেরানো প্রায় অসম্ভব।

দেয়াল

এক ডেটাবেজ, তিন বিদ্যালয় — দেয়াল কীভাবে টানে

মিডলওয়্যার একবার ঠিক করে, প্রতিটি কোয়েরি সেটাই ব্যবহার করে



শিক্ষকের নোট: ডান দিকের লাল বাক্সটি পড়ে জিজ্ঞেস করুন: "এরর না দিয়ে ভুল ডেটা দেওয়া আর এরর দেওয়া — কোনটা বিপজ্জনক?" উত্তর: ভুল ডেটা, কারণ কেউ টের পায় না। এটাই পরের স্লাইডের ভিত্তি।

সিদ্ধান্ত ১

কলাম যোগ করা, না প্যারেন্ট দিয়ে খোঁজা?

একই প্রশ্ন পাঁচটি টেবিলে — উত্তর দুই রকম

যখন কলাম যোগ করলাম

```
# attendance/models.py  (workspace আছে)
qs = AttendanceRecord.objects.filter(
    workspace=request.workspace, date=today)

# lessons/models.py  (Homework-এ workspace আছে)
qs = Homework.objects.filter(
    workspace=ws, class_group=cg, due_at__gte=now)
```

এক JOIN কম, কোয়েরি ছোট, ভুলের সুযোগ কম।

যখন কলাম দিইনি

```
# Mark: workspace কলাম নেই, exam-এ আছে
Mark.objects.filter(
    exam__workspace=ws, exam__is_published=True)

# Chapter: syllabus-এ workspace আছে
Chapter.objects.filter(syllabus__workspace=ws)
```

এক JOIN বেশি, কিন্তু সত্য এক জায়গায়।

প্রশ্ন	কলাম যোগ	প্যারেন্ট-পথ
কখন বেছে নিলাম	টেবিলটা নিজে ঘন ঘন লিস্ট হয়, এবং প্যারেন্ট ছাড়াও কাজে লাগে (`Homework`, `Exam`, `AttendanceRecord`)	টেবিলটা প্যারেন্ট ছাড়া কখনো পড়া হয় না (`Mark`, `Chapter`, `Topic`, `Submission`, `GradeBand`)
ভুলের ধরন	দুই জায়গায় `workspace` — একদিন একটি সারিতে ভুল মান ঢুকে যায়	ভিউতে `workspace` ফিল্টার ভুলে যাওয়া — ডেটা মিশে যায়, এরর নেই
এই ভুল ধরার উপায়	`clean()`-এ প্যারেন্টের workspace মিলিয়ে দেখা	`scoping.py`-র একক হেল্পার + ১০৩ টেস্টের আইসোলেশন টেস্ট

শিক্ষকের নোট: শিক্ষার্থীরা প্রায় সবসময়ই সব টেবিলে `workspace` যোগ করতে চায় — সহজ বলে। তাদের জিজ্ঞেস করুন: "তাহলে Mark-এ workspace আর Exam-এ workspace আলাদা হলে কোনটা সত্য?" দ্বিতীয় সত্য সবসময় হারায়।

সিদ্ধান্ত ২

প্রথম মাইগ্রেশনের আগেই নিজের ইউজার মডেল

একবার ভুল করলে সারাজীবন বয়ে বেড়াতে হয় — তাই একবারেই ঠিক

```
# apps/accounts/models.py
class User(AbstractBaseUser, PermissionsMixin):
    email = models.EmailField(unique=True, db_index=True)
    full_name = models.CharField(max_length=150)
    phone = models.CharField(max_length=16, blank=True,
                             validators=[phone_validator])
    is_active = models.BooleanField(default=True)
    USERNAME_FIELD = "email"
```

`USERNAME_FIELD = "email"` — লগইন আইডি ইমেইল। নাম বদলালে লগইন বদলায় না; `@ims.local` দিয়ে ডেমো চলে, বাস্তবে আসল ইমেইল।

যদি ডিফল্ট `User` দিয়ে শুরু করতাম	পরিণতি
পরে নিজের মডেল দরকার হলে	সব FK ভাঙে — `AUTH_USER_MODEL` বদলানো মানে ৭০টি সম্পর্ক আবার
ইউজারনেম রাখতে হতো	স্কুলে সবাই ইমেইলেই চেনে; ইউজারনেম বানানো বাড়তি কাজ, ভুলের উৎস
ফোন নম্বর আলাদা প্রোফাইলে	আরও একটি টেবিল, আরও একটি JOIN — প্রতিটি SMS-এ

প্রমাণ: `apps/accounts/migrations/0001_initial.py` — প্রজেক্টের প্রথম মাইগ্রেশন, আর সেটিই নিজের মডেল।

শিক্ষকের নোট: এই সিদ্ধান্তটি ডেটাবেজের নয়, শৃঙ্খলার। বলুন: "শেষে ঠিক করা যায়" — ডেটাবেজে এই বাক্যটি প্রায়ই মিথ্যা। যা পরে বদলাতে অসাধ্য, তা প্রথম দিনেই ঠিক করতে হয়।

সিদ্ধান্ত ৩

এক ইউজার, বহু বিদ্যালয়, প্রতি বিদ্যালয়ে ভিন্ন ভূমিকা

ভূমিকা ইউজারের কলামে নয় — `Membership` সারিতে

টেবিল	কী বলে	কেন এখানে
`accounts.User`	"এই ব্যক্তিটি কে" — ইমেইল, নাম, ফোন	পরিচয় গ্লোবাল; স্কুল বদলালেও পরিচয় এক
`workspaces.Membership`	"এই বিদ্যালয়ে তার ভূমিকা কী"	একই মানুষ এক স্কুলে শিক্ষক, আরেক স্কুলে সমন্বয়ক — একটি `role` কলাম দিয়ে সম্ভব নয়
`students.StudentProfile`	"ছাত্র হিসেবে তার নম্বর, ভর্তি নম্বর"	প্রতিটি ছাত্রের অ্যাকাউন্ট থাকে না; থাকলেও প্রোফাইল আলাদা তথ্য
`academics.CoordinatorScope`	"সমন্বয়ক কোন বিভাগ/ক্লাস দেখবে"	অনুমতি ডেটা হিসেবে রাখা — রোল-স্ট্রিং-এ লুকানো নয়

প্রমাণ ১: `Membership`-এ `UniqueConstraint(["user","workspace"])` — এক স্কুলে এক ভূমিকা। দুই ভূমিকা দরকার হলে সেটি ডিজাইনের সংকেত, দুটি সারি নয়।

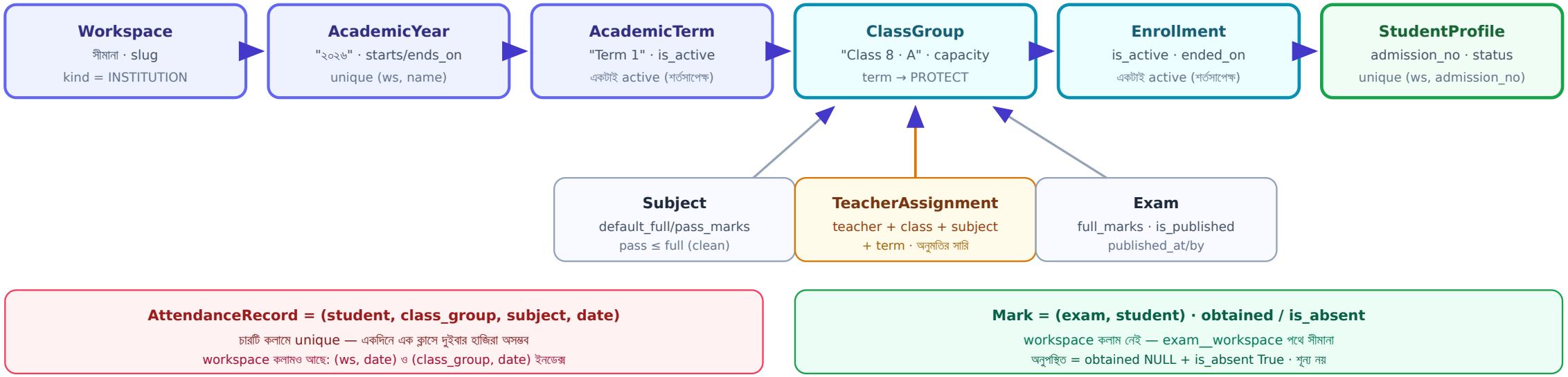
প্রমাণ ২: `StudentProfile.account` ← `OneToOneField(..., on_delete=SET_NULL, null=True)`। অ্যাকাউন্ট মুছলেও ছাত্রের নম্বর-হাজিরা টিকে থাকে।

শিক্ষকের নোট: বাস্তব উদাহরণ দিন: একজন শিক্ষক দুটি স্কুলে পড়ান। তিনি দ্বিতীয় স্কুলে যোগ দিলে নতুন অ্যাকাউন্ট বানাতে হয় না — কেবল একটি `Membership` সারি। এটাই "এক ইউজার, বহু প্রতিষ্ঠান"।

কশিরা

একাডেমিক মেরুদণ্ড — ছয় ধাপের শৃঙ্খল

প্রতিটি রেকর্ড এই শৃঙ্খলের একটি সংযোগ-বিন্দুতে ঝোলে



শিক্ষকের নোট: আঙুল দিয়ে শৃঙ্খল ধরে ধরে বলুন: "উপরের ছয়টি টেবিল না থাকলে নিচের দুটির কোনো অর্থ নেই।" তারপর জিজ্ঞেস করুন — ছাত্র ক্লাস বদলালে কী হয়? উত্তর: নতুন `Enrollment` সারি, পুরনোটি `ended_on` নিয়ে নিষ্ক্রিয়।

নিয়ম ১

`unique_together` — ১৩টি, প্রতিটি একটি বাস্তব ভুল আটকায়

"একই জিনিস দুবার" সমস্যার ডেটাবেজ-স্তরের উত্তর

টেবিল	unique	কোন ভুল আটকায়
StudentProfile	workspace · admission_no	এক স্কুলে একই ভর্তি নম্বর দুজন ছাত্রের নয়
AcademicYear	workspace · name	"২০২৬" দুইবার তৈরি
AcademicTerm	workspace · year · name	এক বছরেই দুইবার "Term 1"
ClassGroup	workspace · term · name · section	"Class 8 · A" দুবার, একই সেশনে
TeacherAssignment	teacher · class_group · subject · term	একই বিষয়ে একজনকে দুইবার যোগ
AttendanceRecord	student · class_group · subject · date	একদিনে এক ছাত্রের দুই হাজিরা
Exam	class_group · subject · term · name	"মিডটার্ম" পরীক্ষা দুইবার
LessonLog · Syllabus · Chapter · Topic · GradeScale · Department · Subject	৩–২ কলাম	পাঠ-ডায়েরি, সিলেবাস ও গ্রেড স্কেলে একই পুনরাবৃত্তি

সবগুলোতেই `workspace` প্রথমে — কারণ uniqueness-ও স্কুলের ভেতরে, স্কুল জুড়ে নয়। দুটি স্কুলের "Class 8 · A" এক হতে পারে, এবং হওয়া উচিত।

শিক্ষকের নোট: শেষ বাক্যটি বোর্ডে লিখুন। শিক্ষার্থীরা প্রথমে `unique=True` দিতে চায় — তখন দ্বিতীয় স্কুল "Class 8 · A" বানাতে পারে না। বাগটা ডেমোতে চোখে দেখা যায় না, দ্বিতীয় স্কুল যোগ করলেই দেখা যায়।

নিয়ম ২

শর্তসাপেক্ষ unique — "একটাই active", ডেটাবেজেই আটকানো

৮টি `UniqueConstraint(condition=…)`; পাইথন নয়, ডেটাবেজ সত্য বলে

```
class AcademicTerm(TimeStamped):
    is_active = models.BooleanField(default=False)

    class Meta:
        constraints = [
            models.UniqueConstraint(
                fields=["workspace"],
                condition=Q(is_active=True),
                name="uniq_active_term_per_workspace",
            )
        ]
```

অর্থ: যেসব সারিতে `is_active=True`, তাদের মধ্যে `workspace` অনন্য। বাকি সারি যত খুশি।

কনস্ট্রেইন্টের নাম	যা অনন্য
`uniq_active_term_per_workspace`	সক্রিয় সেশন
`uniq_active_grade_scale_per_workspace`	সক্রিয় গ্রেড স্কেল
`uniq_active_enrollment_student_class`	ছাত্রের সক্রিয় এনরোলমেন্ট
`uniq_membership_user_workspace`	স্কুলে ইউজারের ভূমিকা
`unique_custom_domain`	কাস্টম ডোমেইন

কেন `clean()` নয়? দুই ইউজার একসাথে সেভ করলে দুটোই `clean()` পাশ করে যেতে পারে — কেবল ডেটাবেজ-স্তরের নিয়মই সত্যিকারের দেয়াল।

শিক্ষকের নোট: "রেস কন্ডিশন" শব্দটি এখানেই প্রথম ব্যবহার করুন — দুই শিক্ষক একই মুহূর্তে সেশন বদলালে কী হয়? পাইথনে চেক করে নয়, ইনডেক্সই শেষ কথা বলে।

নিয়ম ৩

`CheckConstraint` — মিথ্যা তথ্য ঢোকার পথ বন্ধ

২টি check; বাকি সব নিয়ম `clean()`-এ — কেন এই ভাগ

ডেটাবেজে (২টি)

```
# assessments · নাম: grade_band_min_le_max
Q(min_percent__lte=F("max_percent"))

# academics · নাম: coordinator_scope_needs_a_target
~(Q(department__isnull=True) &
  Q(class_group__isnull=True))
```

একটি সারির **নিজের** দুটি কলাম তুলনা — JOIN বা অন্য টেবিল লাগে না।

অ্যাপ্লিকেশনে (`clean()`)

`ends_on > starts_on`	AcademicYear · Term
`pass_marks ≤ full_marks`	Subject · Exam
মার্ক ≤ full_marks · ঋণাত্মক নয়	Mark
অনুপস্থিত ছাত্রের নম্বর নেই	Mark
প্যারেন্ট অন্য workspace-এর নয়	ClassGroup · Term
ভবিষ্যতের তারিখে হাজিরা নয়	AttendanceRecord

সৎ ফাঁক: উপরের ছয়টি নিয়ম কেবল ফর্ম/ভিউ দিয়ে গেলে চলে; সরাসরি `.objects.create()` বা শেল দিয়ে গেলে `clean()` ডাকা হয় না। এই ছয়টির একটিও ডেটাবেজ-স্তরে নেই।

শিক্ষকের নোট: এই ফাঁকটি লুকাবেন না — জিজ্ঞেস করুন "কোন নিয়মটা ডেটাবেজে রাখা উচিত ছিল?" উত্তর: `pass_marks ≤ full_marks` (নিজের দুই কলাম, তাই সহজে সম্ভব ছিল) আর প্যারেন্ট-ওয়ার্কস্পেস মিল (অন্য টেবিল, তাই কঠিন)।

নিয়ম ৪

এনরোলমেন্ট: ইতিহাস মুছি না, নিষ্ক্রিয় করি

শর্তসাপেক্ষ unique-এর সবচেয়ে দরকারি ব্যবহার

```
class Enrollment(TimeStamped):
    student = FK(StudentProfile, CASCADE)
    class_group = FK(ClassGroup, CASCADE)
    is_active = models.BooleanField(default=True)
    ended_on = models.DateField(null=True, blank=True)
    end_reason = CharField(max_length=200)

    class Meta:
        constraints = [UniqueConstraint(
            fields=["student", "class_group"],
            condition=Q(is_active=True),
            name="uniq_active_enrollment_student_class")]
```

ঘটনা	ডেটায় যা ঘটে
ক্লাস বদল	পুরনো সারি: `is_active=False` + `ended_on` + `end_reason`; নতুন সারি তৈরি
একই ক্লাসে ফেরা	নতুন সারি — পুরনোটি ইতিহাসে থেকে যায়
ভুল করে দুইবার active	ডেটাবেজ নিজেই আটকায়, ভিউ নয়
হাজিরা নেওয়া	`clean()` দেখে ছাত্র সত্যিই এই ক্লাসে active কি না

একই প্যাটার্ন তিন জায়গায়: সক্রিয় **সেশন**, সক্রিয় **গ্রেড স্কেল**, সক্রিয় **ভূমিকা**।

দাম: এক ছাত্র পরপর দুই বার একই ক্লাসে ভর্তি হলে ডেটা ঠিকই থাকে, কিন্তু `ended_on` ছাড়া পুরনো সারি নিষ্ক্রিয় করার সুযোগ নেই — ভিউতে অবশ্যই `ended_on` সেট করতে হয়, নাহলে ইতিহাসে ফাঁক থেকে যায়।

শিক্ষকের নোট: বাস্তব প্রশ্ন করুন: "ছাত্র ক্লাস বদলাল, রিপোর্ট কার্ডে পুরনো ক্লাসের নম্বর কোথায় থাকবে?" উত্তর — পুরনো এনরোলমেন্ট সারিতে, কারণ নম্বর কখনো সরানো হয় না।

গতি

১৩টি হাতে লেখা ইনডেক্স — কোন কোয়েরির জন্য

মোট ইনডেক্স ≈ ১৩৯; তার ১২৬টি Django নিজেই বানায়

ইনডেক্স	যে কোয়েরি দ্রুত হয়
`(workspace, date)` · Attendance	আজকের হাজিরা, স্কুলের সব ক্লাস
`(class_group, date)`	এক ক্লাসের দৈনিক হাজিরা
`(workspace, class_group, is_active)`	ক্লাসের সক্রিয় ছাত্র-তালিকা
`(workspace, status)` · Student	সক্রিয় ছাত্রের তালিকা
`(workspace, term, is_archived)`	সেশনের ক্লাস-তালিকা
`(workspace, term, is_published)`	প্রকাশিত ফলাফল
`(workspace, class_group, due_at)`	আসন্ন হোমওয়ার্ক
`(workspace, -created_at)` · Audit	সাম্প্রতিক অডিট লগ
`(user, is_active)` · Membership	ইউজারের স্কুল-তালিকা (প্রতি রিকোয়েস্টে)

৩১ · প্রাইমারি কী

স্বয়ংক্রিয়

৭৪ · ফরেন কী

Django প্রতিটি FK-তে

২১ · unique নিয়ম

১৩ unique_together + ৮ শর্তসাপেক্ষ

১৩ · হাতে লেখা

কেবল আসল কোয়েরি দেখে

এগুলোর কোনোটিই `EXPLAIN` দেখে নেওয়া নয় — লিস্ট-ভিউয়ের `filter()` পড়ে অনুমান করা। প্রকৃত পরিমাপ এখনো হয়নি।

শিক্ষকের নোট: সততাটুকু জোর দিয়ে বলুন: ইনডেক্স "সঠিক" প্রমাণ করতে `EXPLAIN ANALYZE` লাগে, অনুমান নয়। ৪০০ ছাত্রের ডেমোতে পার্থক্যই দেখা যায় না; পার্থক্য দেখা যায় ৫০ হাজার সারিতে।

সিদ্ধান্ত ৪

কিছু মুছলে কী হয় — ৭০টি FK-র তিন রকম উত্তর

CASCADE ৪৮ · SET_NULL ১৬ · PROTECT ৬ — আর ৪টি OneToOne

CASCADE · ৪৮

প্যারেন্ট গেলে সন্তানও যায়। **কেবল তখনই**, যখন সন্তানের স্বাধীন অস্তিত্ব নেই।

ClassGroup মুছলে তার Exam, AttendanceRecord, Homework — সব। বিদ্যালয় মুছলে পুরো স্কুলের ডেটা।

SET_NULL · ১৬

রেফারেন্স মুছে যায়, তথ্য থাকে — "কে করেছিল" ঘরটি ফাঁকা হয়।

১৬টির মধ্যে **১২টিই ইউজার**: `recorded_by`, `entered_by`, `published_by`, `reviewed_by` ...

PROTECT · ৬

মুছতেই দেওয়া হয় না — `ProtectedError` ওঠে। রক্ষা করতে চাইলে **আগে** সিদ্ধান্ত নিতে হয়।

টার্ম, প্ল্যান, ইনভয়েস, স্কুলের মালিক — এই চার ধরনের জিনিস।

নিয়ম: **যে সম্পর্কে অর্থ হারায়, সেখানে CASCADE; যে সম্পর্কে কেবল দায়িত্ব হারায়, সেখানে SET_NULL; যে সম্পর্কে হিসাব ভাঙে, সেখানে PROTECT।**

শিক্ষকের নোট: শিক্ষার্থীদের সবসময় CASCADE লিখতে অভ্যাস। তিনটি উদাহরণ জিজ্ঞেস করুন: "শিক্ষক চাকরি ছাড়লে তার নেওয়া হাজিরা মুছবে?" — না, তাই SET_NULL। "টার্ম মুছলে তার ক্লাস?" — না, PROTECT।

PROTECT

যে ছয়টি জিনিস ভুল করে মুছে ফেলা যাবে না

প্রতিটির পেছনে একটি বাস্তব দুঃস্বপ্ন

FK	মুছতে গেলে	কেন এটাই সঠিক
`Workspace.owner` → User	`ProtectedError`	মালিকহীন স্কুল = কেউ ঢুকতে পারে না, কেউ মেটাতে পারে না
`ClassGroup.term` → AcademicTerm	আটকে যায়	সেশন মুছলে তার ক্লাস, হাজিরা, পরীক্ষা — সব অনাথ
`Exam.term` → AcademicTerm	আটকে যায়	ফলাফল কোন সেশনের, তার প্রমাণ হারায়
`Subscription.plan` → Plan	আটকে যায়	বিলিং ইতিহাসে দামের ভিত্তি অদৃশ্য হয়ে যায়
`Invoice.workspace` → Workspace	আটকে যায়	টাকা নেওয়ার রেকর্ড কখনো "মুছে ফেলা" যায় না
`Invoice.plan` → Plan	আটকে যায়	কোন প্ল্যানে কত টাকা — ইনভয়েস নিজেই সত্য বলে

দাম: প্ল্যানের দাম বদলাতে চাইলে পুরনো প্ল্যান মুছবেন না — নতুন প্ল্যান বানিয়ে `is_active=False` করুন।

টেস্ট: এই ছয়টির একটিও আলাদা টেস্টে লেখা নেই — `PROTECT` Django-র নিজের আচরণ, তাই বাড়তি টেস্ট লেখা হয়নি।

শিক্ষকের নোট: "কেন মালিক PROTECT, অথচ হাজিরা-লেখক SET_NULL?" — এই প্রশ্নটিই পাঠের মূল। উত্তর: মালিক ছাড়া স্কুল চলে না; হাজিরা-লেখক ছাড়া হাজিরা চলে।

SET_NULL

"কে করেছিল" ঘরটি ফাঁকা হয়, তথ্য থাকে

১৬টি SET_NULL; ১২টি ইউজারের দিকে — একটি ইচ্ছাকৃত প্যাটার্ন

```
recorded_by = models.ForeignKey(
    settings.AUTH_USER_MODEL,
    on_delete=models.SET_NULL, null=True,
    related_name="+")

updated_by = models.ForeignKey(
    settings.AUTH_USER_MODEL,
    on_delete=models.SET_NULL, null=True,
    blank=True, related_name="+")
```

হাজিরা সারি থেকে যায়; কে লিখেছিল, তা আর দেখা যায় না।

SET_NULL (১২টি ইউজার)	কেন
`AttendanceRecord.recorded_by` · `updated_by`	হাজিরা ছাত্রের তথ্য, শিক্ষকের নয়
`Mark.entered_by` · `Exam.published_by`	নম্বর থাকা চাই, লেখক বদলাতে পারে
`AuditLog.actor`	লগ থাকে, ইউজার না-ও থাকতে পারে
`Enrollment.enrolled_by` · `Homework.created_by`	ভর্তি ইতিহাস মুছবে না

ফাঁক: ইউজার মুছে গেলে "কে করেছিল" কেবল `AuditLog`-এ থাকে — আর সেটি সব কাজে লেখা হয় না।

শিক্ষকের নোট: প্রশ্ন করুন: "শিক্ষক অবসরে গেলে তার ৬ বছরের হাজিরা মুছে ফেলা কি ঠিক?" উত্তর স্পষ্ট না হলেও আলোচনাটাই শেখায় — ডেটাবেজ ডিজাইনে সবসময় "একটি সারি" নয়, "একটি মানুষের কাজের ইতিহাস" জড়িত।

কেস ১

একটি হাজিরা সারির ১৩টি ঘর — কেন প্রতিটি দরকার

attendance.AttendanceRecord · ১৩ কলাম · ১ unique · ২ ইনডেক্স

কলাম	কেন আছে
`workspace`	লিস্ট-কোয়েরির একক শর্ত
`student` · `class_group`	"কোন ছাত্র, কোন ক্লাসে"
`subject`	বিষয়ভিত্তিক হাজিরা; একই দিনে ৬ বিষয়
`date`	দিন; ভবিষ্যতের তারিখ নিষিদ্ধ (`clean()`)
`status`	PRESENT · ABSENT · LATE · LEAVE
`note`	২০০ অক্ষরের টীকা

কলাম	কেন আছে
`recorded_by`	কে প্রথম লিখল (SET_NULL)
`updated_by`	কে সংশোধন করল — না থাকলে সংশোধন নয়
`correction_reason`	সংশোধনের কারণ, ২০০ অক্ষরে
`created_at` · `updated_at`	কখন লেখা, কখন বদলানো
`id`	সারির পরিচয়

সংশোধনের অনুমতি কার, কত দিন পর্যন্ত — `AttendancePolicy.correction_window_days = 7` ; ন্যূনতম উপস্থিতি ৭৫%। নীতি ডেটায়, কোডে নয়।

খেয়াল করুন: সংশোধন মানে পুরনো সারি বদলানো, নতুন সারি নয়। তাই ইতিহাস এক সারিতেই — কে, কখন, কেন বদলাল, তিনটিই ওই সারির ভেতরে।

শিক্ষকের নোট: প্রশ্ন করুন: "শিক্ষক ভুল করে সবাইকে অনুপস্থিত দিয়ে ফেললেন — ডেটাবেজে কী ঘটবে?" দুটি উত্তর শোনান: বদলে ফেলা (ইতিহাস হারায়) আর নতুন সারি (ডুপ্লিকেট)। প্রকল্পে তৃতীয় পথ: একই সারি + `updated_by` + `correction_reason` ।

কেস ২

অনুপস্থিত আর শূন্য নম্বর — ডেটায় আলাদা, তাই আলাদা ফল

assessments.Mark · ৮ কলাম · (exam, student) unique

```
obtained = DecimalField(max_digits=6, decimal_places=2,
                        null=True, blank=True)
is_absent = models.BooleanField(default=False)

def clean(self):
    if self.is_absent and self.obtained is not None:
        raise ValidationError(...)     # দুটোই সত্য নয়
    if self.obtained > self.exam.full_marks:
        raise ValidationError(...)
```

অবস্থা	ডেটা	রিপোর্টে
উপস্থিত, ০ পেয়েছে	`obtained=0`, `is_absent=False`	গড়ে ০ ধরা হয়, ফেল
অনুপস্থিত	`obtained=NULL`, `is_absent=True`	গড় থেকে বাদ, আলাদা গণনা
নম্বর এখনো তোলা হয়নি	`NULL`, `is_absent=False`	গড় থেকে বাদ

যদি অনুপস্থিতকে ০ লিখতাম, ক্লাসের গড় আর পাশের হার দুটোই মিথ্যা হতো — আর কেউ ধরতে পারত না।

গণনার নিয়ম এক জায়গায়: `build_result_sheet(exam)` — শুধু পড়ে, লেখে না। তাই হাজিরা-রিপোর্ট আর রেজাল্ট-শিট একই ফাংশন ডাকে, দুটি আলাদা হিসাব হয় না।

শিক্ষকের নোট: এই স্লাইডটিই "NULL-এর অর্থ" শেখানোর সেরা জায়গা। তিন ধরনের অনিশ্চয়তা দেখান: অজানা (NULL), শূন্য (০), প্রযোজ্য নয় (অনুপস্থিত) — তিনটিই আলাদা।

কেস ৩

ফলাফল প্রকাশ — এক বুলিয়ান নয়, একটি তারিখও

is_published · published_at · published_by · তিনটিই দরকার

কলাম	উত্তর দেয়
`is_published`	ছাত্র এখন দেখতে পাবে কি না
`published_at`	কখন প্রকাশ হলো — অভিযোগের সময় কাজে লাগে
`published_by`	কে প্রকাশ করল (SET_NULL, `related_name="+"`)

```
exam.is_published = True
exam.published_at = timezone.now()
exam.published_by = request.user
exam.save(update_fields=[
    "is_published", "published_at",
    "published_by", "updated_at"])
```

প্রশ্ন: শুধু `is_published` রাখলে কী হারাতাম? উত্তর: "কবে, কে" — এবং প্রতারণার অভিযোগ এলে কোনো প্রমাণ থাকত না।

প্রবেশ-নিয়ন্ত্রণ দুই জায়গায়: ভিউতে ভূমিকা (`TEACHING`) + `teacher_can_access_class()`; আর ছাত্রের পথে অপ্রকাশিত ফল হলে `PermissionDenied` → ৪০৩।

সৎ ফাঁক: প্রকাশের সময় কোনো `transaction.atomic` নেই — একটি সারি বদলানোয় দরকারও নেই; কিন্তু `log()` আর `save()` আলাদা লেনদেন, তাই লগ লেখা ব্যর্থ হলে (নীরবে) থেকেই যায়।

শিক্ষকের নোট: বাস্তব ঘটনা বলুন: অভিভাবক এসে বললেন "গতকাল রাতে নম্বর বদলেছে"। `published_at` + অডিট লগ থাকলে উত্তর দেওয়া যায়; না থাকলে কেবল অনুমান।

কেস ৪

টাকা কখনো `float` নয় — ৬টি `DecimalField`

আর ইনভয়েস নম্বরের একটি বাস্তব দুর্বলতা

কলাম	সেটিং
`Plan.price`	max_digits=10 · decimal_places=2 · ≥ ০
`Invoice.amount`	max_digits=10 · decimal_places=2
`Mark.obtained`	max_digits=6 · decimal_places=2 · NULL অনুমোদিত
`GradeBand.min/max_percent`	max_digits=5 · decimal_places=2
`GradeBand.gpa`	max_digits=3 · default 0.00

`float`-এ ০.১ + ০.২ = ০.৩০০০০০০০০০০০০০০৪। এক বছরে হাজার ইনভয়েসে সেটি টাকায় পরিণত হয়।

```
@staticmethod
def _next_number():
    year = timezone.localdate().year
    prefix = f"{settings.INVOICE_PREFIX}-{year}-"
    last = (Invoice.objects
            .filter(number__startswith=prefix)
            .order_by("-number")
            .values_list("number", flat=True).first())
    n = int(last.split("-")[-1]) + 1 if last else 1
    return f"{prefix}{n:04d}"
```

দুর্বলতা: "শেষটি পড়ে এক যোগ" — একই মুহূর্তে দুই ইনভয়েস হলে দুটোই একই নম্বর পায়। `number`-এ `unique=True` থাকায় ডেটাবেজ আটকায়, কিন্তু ব্যবহারকারী ৫০০ এরর দেখে।

সঠিক পথ হতো ডেটাবেজ সিকোয়েন্স, অথবা `select_for_update()` দিয়ে সারি লক করে নম্বর বাড়ানো। এখনো করা হয়নি — এটি প্রকৃত, চিহ্নিত ঋণ।

শিক্ষকের নোট: এই স্লাইডে সততাই মূল্য: কোড কাজ করে, কিন্তু একই সময়ে দুই ইনভয়েস হলে ভাঙে। শিক্ষার্থীদের জিজ্ঞেস করুন — "লোড কম, তাই ধরা পড়েনি" এটা কি যথেষ্ট কারণ?

কেস ৫

সময়ের ৫৩টি ঘর — কে লেখে, কে বদলায়

১৯টি মডেল `TimeStamped` থেকে উত্তরাধিকার পায় (১৯ × ২ = ৩৮ ঘর)

```
class TimeStamped(models.Model):
    created_at = DateTimeField(auto_now_add=True)
    updated_at = DateTimeField(auto_now=True)

    class Meta:
        abstract = True
```

ফিল্ড	কে লেখে
`auto_now_add`	প্রথম সেভে একবার; পরে কেউ বদলাতে পারে না
`auto_now`	প্রতিটি `save()`-এ আবার
`default=timezone.now`	মডেল-স্তরে নয়, মাইগ্রেশনে মান লেখা হয়

যেখানে সময় গুরুত্বপূর্ণ	কেন
`AttendanceRecord.date` (`DateField`)	হাজিরার দিন — সময় নয়, তারিখ
`Homework.due_at` (datetime)	"আজ রাত ১১:৫৯" — সময় দরকার
`AuditLog.created_at`	সাম্প্রতিক লগ ইনডেক্সের ভিত্তি
`Invoice.issued_on`	`timezone.localdate()` — সার্ভার UTC নয়, স্থানীয় দিন

`USE_TZ=True` · ডেটাবেজে UTC · পর্দায় `Asia/Dhaka`। ভুল করলে "আজকের হাজিরা" রাত ১২টার পর একদিন পিছিয়ে যায়।

নিয়ম: **যে তথ্য ব্যবহারকারী বেছে নেয়, তার জন্য `default`; যে তথ্য সিস্টেম লেখে, তার জন্য `auto_now*`।** দুটো মিশিয়ে ফেললেই পরীক্ষায় সময় নিয়ে বিভ্রান্তি।

শিক্ষকের নোট: প্রশ্ন করুন: "মাইগ্রেশনে `auto_now_add` কেন সমস্যা করে?" — কারণ পুরনো সারিতে তারিখ লাগে, আর `auto_now_add` মাইগ্রেশনকে সেটি লিখতে দেয় না; তখন `default` দিয়ে দুই ধাপে করতে হয়।

প্রমাণ

কোন টেস্ট কোন ডেটাবেজ-নিয়ম প্রমাণ করে

১০৩টি টেস্ট · এর মধ্যে ডেটা-নিয়মের প্রমাণ ১৪টি

টেস্টের নাম (আসল)	যে নিয়ম প্রমাণ করে	স্তর
`test_only_one_term_can_be_active`	`uniq_active_term_per_workspace`	ডেটাবেজ
`test_saving_twice_does_not_duplicate`	`uniq_attendance_student_class_subject_date`	ডেটাবেজ
`test_domain_cannot_be_stolen_by_a_second_institution`	`unique_custom_domain`	ডেটাবেজ
`test_absent_student_cannot_have_a_mark`	Mark — অনুপস্থিত + নম্বর একসাথে নয়	`clean()`
`test_mark_above_full_marks_is_rejected`	Mark — পূর্ণমানের সীমা	`clean()`
`test_overlapping_bands_are_rejected`	GradeBand — ওভারল্যাপ নয়	`clean()`
`test_unenrolled_student_cannot_be_marked`	AttendanceRecord ↔ Enrollment	`clean()` + কোয়েরি
`test_future_date_is_rejected`	ভবিষ্যতের হাজিরা নয়	`clean()`
`test_invoice_numbers_are_sequential`	ইনভয়েস নম্বরের ধারা	এক-থ্রেডে মাত্র
`test_used_seats_come_from_real_rows`	আসন গোনা হয় সারি থেকে, কাউন্টার থেকে নয়	কোয়েরি

শেষ সারিটি মনে রাখুন: `test_invoice_numbers_are_sequential` ধারাবাহিকতা প্রমাণ করে, কিন্তু **সমান্তরালতায়** ভাঙে কি না তা পরীক্ষা করে না।

শিক্ষকের নোট: টেস্টের নাম পড়ে বলুন — ভালো টেস্টের নামই ডকুমেন্টেশন। "কোন নিয়মটি সত্যিই প্রমাণিত, আর কোনটি নিছক অনুমান" — এই প্রশ্নটি শিক্ষার্থীদের নিজের প্রজেক্টে করতে দিন।

কোয়েরি

N+1 সমস্যা, আর ৫৭টি `select_related`

৩১ টেবিল থাকলেই ধীর হয় না — JOIN হারিয়ে গেলেই ধীর হয়

৪০ ছাত্রের হাজিরা, খারাপ পথ

```
for r in AttendanceRecord.objects.filter(
        workspace=ws, date=today):
    print(r.student.full_name)    # প্রতি সারিতে ১ কোয়েরি
```

১ + ৪০ = ৪১ কোয়েরি

ভালো পথ

```
qs = (AttendanceRecord.objects
      .filter(workspace=ws, date=today)
      .select_related("student", "subject"))
```

২ কোয়েরি — ৪১ নয়

৫৭ · `select_related`
সরাসরি FK — একই SQL-এ JOIN

২ · `prefetch_related`
উল্টো দিক / many-to-many — আলাদা কোয়েরি, তারপর জোড়া

৭ · `Count()`
গোনা ডেটাবেজে, পাইথনে নয়

৩ · `aggregate()` · ২ · `annotate()`
রিপোর্টের গড়/সর্বোচ্চ

সৎ ফাঁক: কোনো কোয়েরি-সংখ্যা কখনো **মাপা হয়নি** — django-debug-toolbar নেই, `connection.queries` কোথাও ব্যবহৃত নয়। ৫৭টি `select_related` লেখা হয়েছে কোড পড়ে, প্রোফাইল করে নয়।

শিক্ষকের নোট: বোর্ডে ১ + ৪০ লিখুন, তারপর প্রশ্ন করুন: "শিক্ষার্থী ৪০০ হলে?" — ৪০১ কোয়েরি। এটাই N+1; নামটাই বলে দেয় কেন: N সারি, প্রতিটির জন্য আরও ১।

ইতিহাস

১২টি মাইগ্রেশন — ডেটাবেজের ডায়েরি

৯ অ্যাপে `0001_initial` · ৩ অ্যাপে `0002` · core ও reports-এ শূন্য

ফাইল	কী যোগ করল
accounts/0001 · academics/0001 · students/0001 · attendance/0001 · lessons/0001 · assessments/0001 · audit/0001 · workspaces/0001	প্রথম স্কিমা (৮টি অ্যাপের মূল টেবিল)
billing/0001	প্ল্যান · সাবস্ক্রিপশন · ইনভয়েস
workspaces/0002_workspacebranding	লোগো · রঙ · কাস্টম ডোমেইন
academics/0002_coordinatorscope	সমন্বয়কের পরিধি
billing/0002_seed_plans_and_grandfather	**ডেটা** মাইগ্রেশন — ৩ প্ল্যান + পুরনো স্কুলের সাবস্ক্রিপশন

```
def forwards(apps, schema_editor):
    Plan = apps.get_model("billing", "Plan")
    ...
    # নতুন স্কুল "চিরকাল ফ্রি" হয়ে যাওয়া
    # ঠেকাতে প্রতিটি workspace-এ
    # একটি Subscription সারি বানানো হয়
```

নিয়ম: স্কিমা মাইগ্রেশন আর ডেটা মাইগ্রেশন আলাদা ফাইলে। একসাথে করলে রোলব্যাকের সময় ডেটা হারায়।

সৎ ফাঁক: `makemigrations --check` বা মাইগ্রেশন-লিন্টার কোথাও স্বয়ংক্রিয় নয়; সিআই নেই। ভুলে মডেল বদলে মাইগ্রেশন না বানালে ধরা পড়বে কেবল ডিপ্লয়ে করে।

শিক্ষকের নোট: জিজ্ঞেস করুন: "মাইগ্রেশন ফাইল হাতে লেখা যায়?" — যায়, এবং কখনো দরকার হয় (ডেটা রূপান্তর)। কিন্তু স্কিমার জন্য কখনো হাতে লিখবেন না; `makemigrations`-এর আউটপুট পড়ুন, তারপর বিশ্বাস করুন।

নিরাপত্তা

ব্যাকআপ — স্কিমা যত ভালো, হারানো ডেটা তত বেদনাদায়ক

`ops/backup.sh` · `ops/restore.sh` · দুটোই ২০ লাইনের কম

```
STAMP=$(date +%Y%m%d-%H%M%S)
pg_dump -Fc -f "$DEST/ims-$STAMP.dump"
tar -czf "$DEST/media-$STAMP.tar.gz" media/
find "$DEST" -name '*.dump'  -mtime +14 -delete
find "$DEST" -name '*.tar.gz' -mtime +14 -delete
```

দুটি জিনিস আলাদা: **ডেটাবেজ** (pg_dump) আর **ফাইল** (media/)। একটি ডাম্প দুইয়ের কোনোটিকেই ঢাকে না।

সিদ্ধান্ত	কারণ
`-Fc` ফরম্যাট	কাস্টম ফরম্যাট — `pg_restore` দিয়ে টেবিল ধরে ধরে ফেরানো যায়
১৪ দিন রাখা	দুই সপ্তাহের মধ্যে ভুল ধরা পড়ে; তার বেশি রাখলে জায়গা নষ্ট
`set -euo pipefail`	মাঝপথে ব্যর্থ হলে স্ক্রিপ্ট থামে, আধা-ব্যাকআপ নয়

সৎ ফাঁক: ব্যাকআপ **একই সার্ভারে** থাকে (`./backups`) — সার্ভার হারালে ব্যাকআপও হারায়। অফসাইট কপি নেই, রিস্টোর কখনো স্বয়ংক্রিয়ভাবে পরীক্ষা করা হয়নি, WAL আর্কাইভ/PITR নেই।

নিয়ম: **যে ব্যাকআপ কখনো ফেরানো হয়নি, সেটি ব্যাকআপ কি না জানা নেই।** তাই পরের কাজ — মাসে একবার রিস্টোর-অনুশীলন।

শিক্ষকের নোট: প্রশ্ন করুন: "১৪ দিনের পুরনো ফাইল মুছে যায় — কোন অবস্থায় সেটি বিপদ?" উত্তর: কেউ দুই সপ্তাহ ধরে ভুল ডেটা লিখলে, আর সেটি তখনই ধরা পড়লে — ভালো ব্যাকআপও ইতিমধ্যে মুছে গেছে।

সততা

এই ডেটাবেজে যা নেই — জেনে রাখুন

আটটি ফাঁক, প্রতিটির সঙ্গে "কীভাবে ধরা পড়বে"

যা নেই	ফল
`EXPLAIN ANALYZE`	ইনডেক্সগুলো অনুমানে বাছা; ধীর কোয়েরি অজানা
সারি-সংখ্যার পরিমাপ	স্কেল কত বড়, তার কোনো হিসাব নেই
Row Level Security	সীমানা অ্যাপে, ডেটাবেজে নয়
রেপ্লিকা / শার্ডিং	একটাই ডেটাবেজ, একটাই সার্ভার

যা নেই	ফল
৮ টেবিলে `workspace` কলাম	প্যারেন্ট-পথে শর্ত; ভুলে গেলে ডেটা মিশে
ইনভয়েস নম্বরে লক	সমান্তরাল সেভে `IntegrityError`
`clean()`-এর DB সংস্করণ	শেল/স্ক্রিপ্ট নিয়ম এড়াতে পারে
মাইগ্রেশন সিআই	মাইগ্রেশন ভুলে যাওয়া ধরা পড়ে ডিপ্লয়ে গিয়ে

৫টি সিদ্ধান্ত যা ঠিক আছে: workspace কলাম · শর্তসাপেক্ষ unique · SET_NULL বনাম PROTECT · ডেটা মাইগ্রেশন · অনুপস্থিত ≠ ০

৫টি কাজ যা এখনো বাকি: EXPLAIN · ইনভয়েস লক · RLS নয়তো টেস্ট-জাল · অফসাইট ব্যাকআপ · মাইগ্রেশন চেক

শিক্ষকের নোট: এই স্লাইডটি মুখস্থ করাতে হবে না — শুধু দেখান যে সৎ তালিকা দিলে বিশ্বাস বাড়ে, কমে না। শেষে জিজ্ঞেস করুন: "এর মধ্যে একটি ফাঁক আজ ঠিক করতে বললে কোনটি বাছতেন?" — উত্তর সাধারণত ইনভয়েস লক।

২

D2 — স্ট্যাক ও সংযোগ

ডেটাবেজ আছে, এখন সেটি কোন তার দিয়ে পর্দায় পৌঁছায় · ২৮ স্লাইড